

Using *Splunk* to Investigate a brute-force

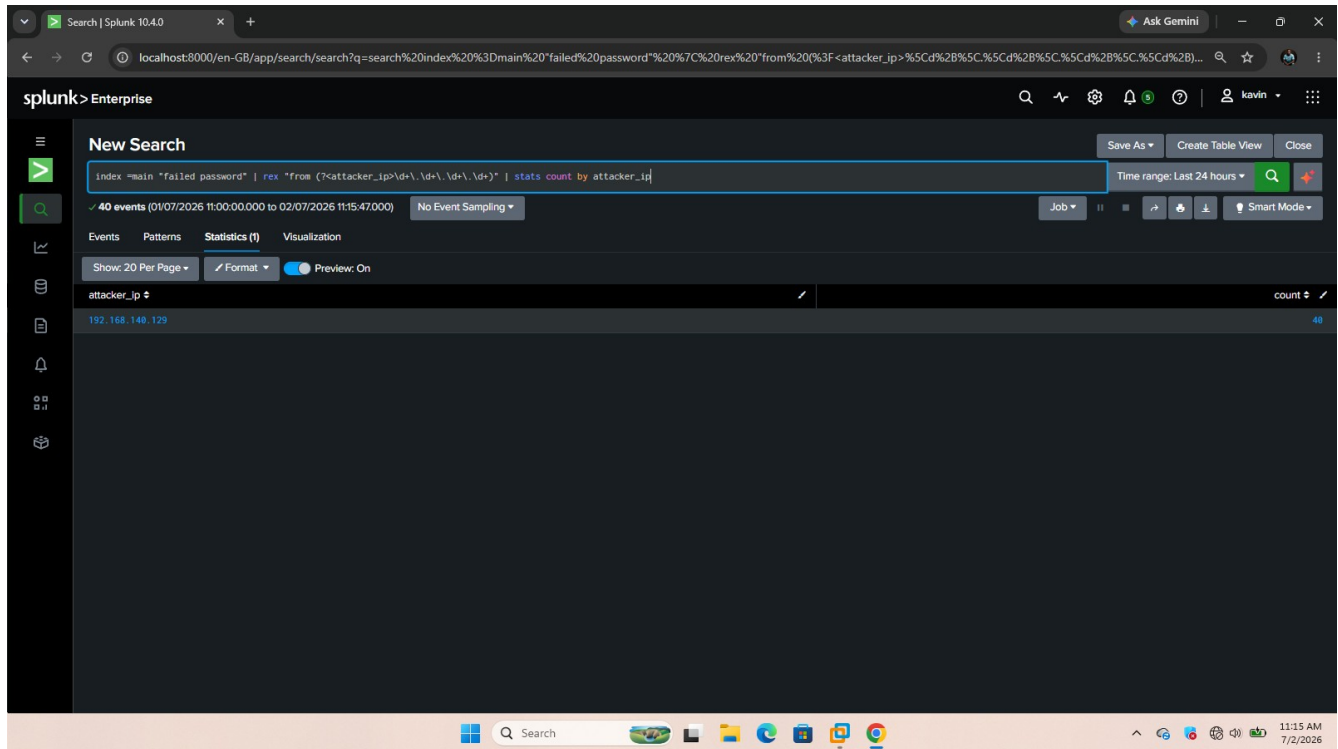
The screenshot displays the Splunk Search interface. The search bar at the top contains the query: `search%20index%20%3D%20main&display.page.search.mode=smart&dispatch.sample_ratio=1&workload_pool=&earliest=-24h%40h&latest=now&sid=1782978074.33`. The results are shown in a table with columns for Time and Event. The events are filtered to show only those from the host 'kavin'.

Time	Event
02/07/2026 10:40:00.191	2026-07-02T07:40:00.191300+00:00 kavin system[1]: Starting sysstat-collect.service - system activity accounting tool...
02/07/2026 10:39:47.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:39:42.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:39:37.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:39:32.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:39:27.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:39:22.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:39:17.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:39:13.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:39:08.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:39:02.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:38:58.000	Brute force attack detected from 192.168.140.129 host = kavin index = main source = alertSsh BruteForce sourcetype = generic_single_line timestamp = none
02/07/2026 10:38:53.443	2026-07-02T07:38:53.443711+00:00 kavin sshd-session[2604]: Connection closed by authenticating user kavin 192.168.140.129 port 35434 [preauth] date_hour = 7 date_mday = 2 date_minute = 38 date_month = July date_second = 53 date_wday = Thursday date_year = 2026 date_zone = 0 host = kavin index = main pid = 2604 process = sshd-session source = /var/log/auth.log sourcetype = syslog src_ip = 192.168.140.129 timeendpos = 32 timestamppos = 0

Here we can see that we have a bruteforce attack that is happening on system or host named kavin.

*Since I have already created an alert here we see that *splunk* already shows a presence of a bruce-force from the specif IP*

Using *Splunk* to Investigate a brute-force



The screenshot shows the Splunk Enterprise web interface. The search bar contains the query: `index="main" "failed password" | rex "from (?<attacker_ip>\d+\.\d+\.\d+\.\d+)" | stats count by attacker_ip`. The search results show 40 events. The table view displays the following data:

attacker_ip	count
192.168.148.129	40

We used | *rex to extract fields from row logs.*

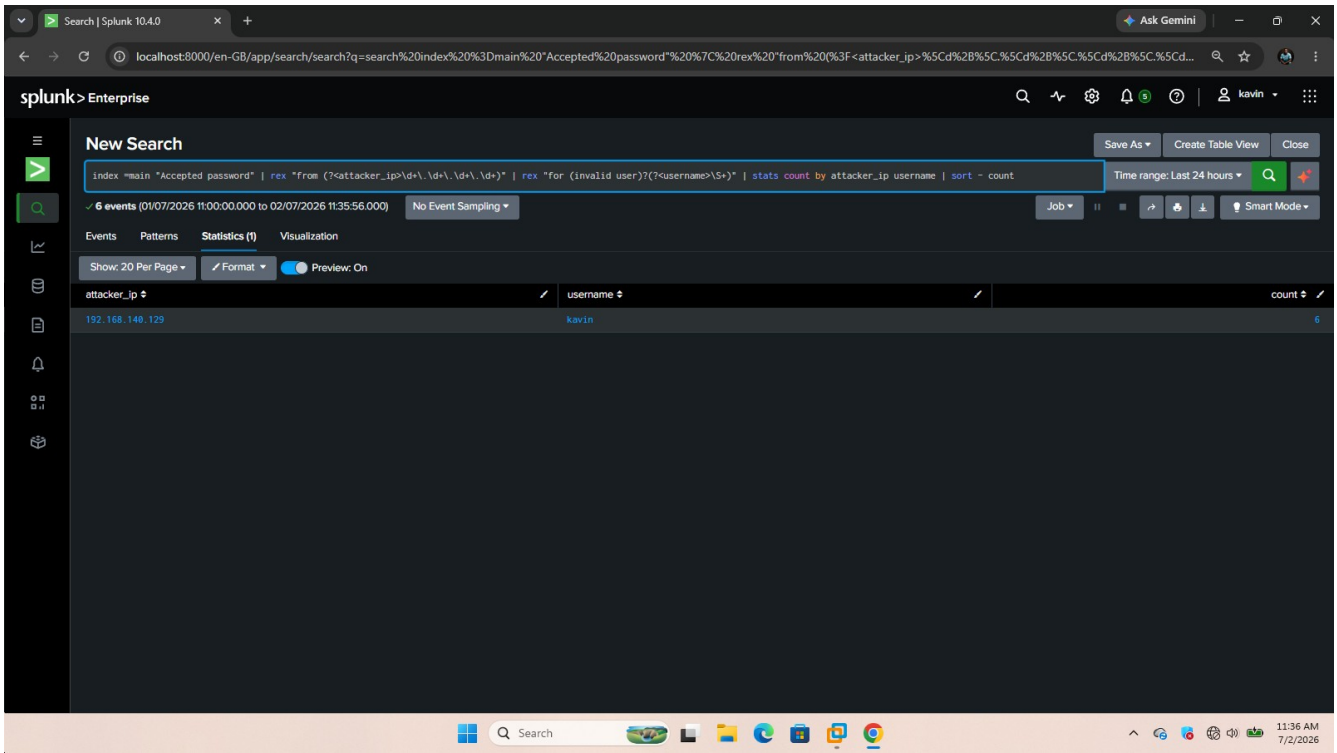
Good to know that /d means (0 – 9) so + means one more of the previous thing so \d+ =192 while \=(.)dot hence it extracts our ip clearly

Then ?>attacker_ip> its used to match our ip with that name so that we can use field “attacker_ip” in our search and tables and even alerts later.

We see on the end that we have count which is displayed by command **stats count by** . We have 40 attempts that could signal a compromised account.

Next question is was there any successful logins?lets go on and see also who does yhis account belong to

Using *Splunk* to Investigate a brute-force



The screenshot shows the Splunk Enterprise search interface. The search bar contains the following query: `index=main "Accepted password" | rex "from (<attacker_ip>[d+].[d+].[d+].[d+])" | rex "for (invalid user)?(<username>[S+])" | stats count by attacker_ip username | sort - count`. The search results show 6 events. The table below displays the results:

attacker_ip	username	count
192.168.148.129	kavin	6

Now here is the **smoking gun** we have changed our search to accepted search shows that the account had 6 accepted logins.

Lets now see how our report would look like:

*Using **Splunk** to Investigate a brute-force*

Report Analysis:

Investigation

Failed Logins

Successful Logins

Source IP

Username

Results

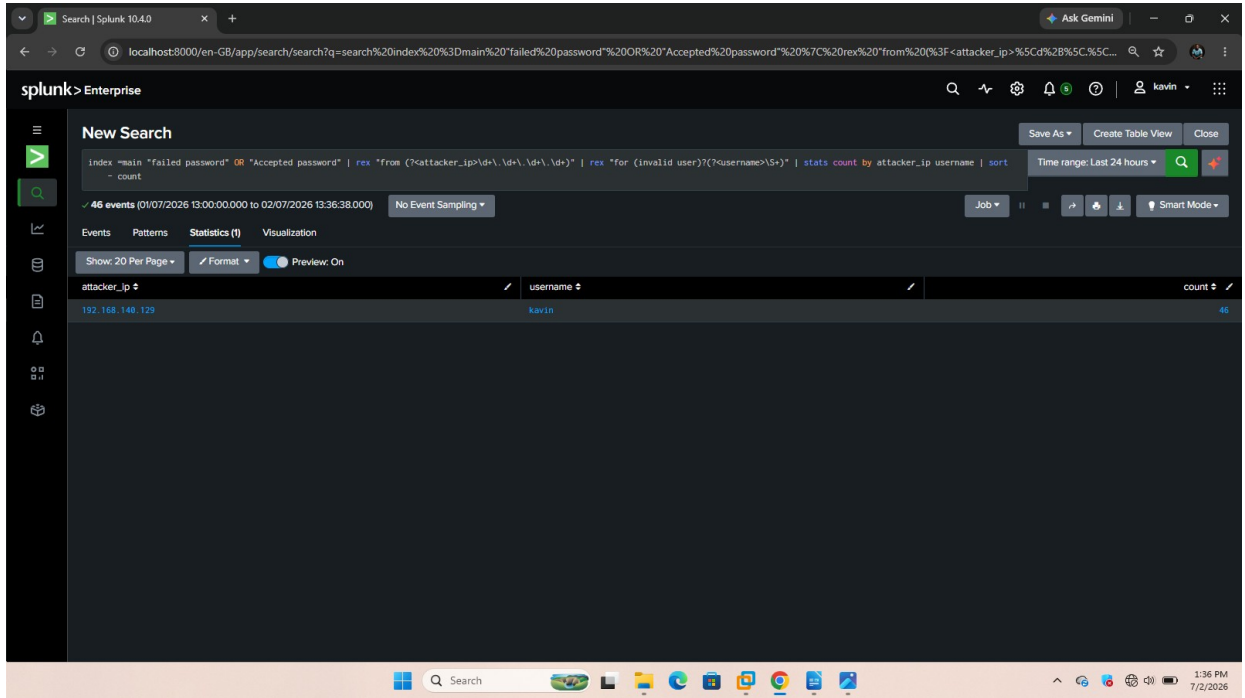
40

6

192.168.140.129

kavin

Using *Splunk* to Investigate a brute-force



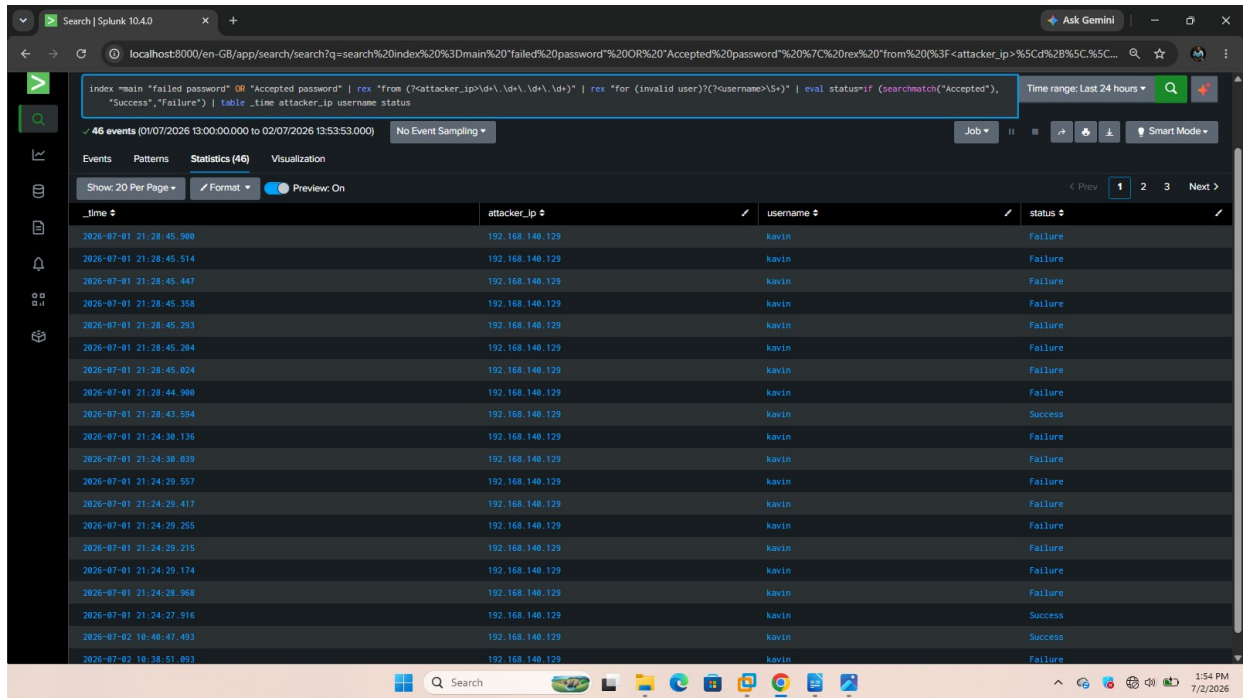
The screenshot shows the Splunk Enterprise web interface. The search bar contains the following query: `index=main "failed password" OR "Accepted password" | rex "from (?<attacker_ip>\d+\.\d+\.\d+\.\d+)" | rex "for (invalid user)?(?<username>\S+)" | stats count by attacker_ip username | sort`. The search results show 46 events. The table below displays the results:

attacker_ip	username	count
192.168.148.129	kavin	46

Here we build a Correlation Search:

And we see our total event is 46

Using Splunk to Investigate a brute-force



Now a new command **eval** :

Eval creates a new field like in our query `status=if (searchmatch("Accepted"), "Success", "Failure")`

If the log contains *Accepted status is Successful*
Otherwise *status is failure*

***_time* is for showing exact time**

Next I create a dashboard:

Using *Splunk* to Investigate a brute-force

Kavin_SOC_Monitoring_Lab_
My Own full Security Operation Center On My Full Lab

SSH Login Status

_time	attacker_ip	username	status
2026-07-01 21:28:45.900	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.514	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.447	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.358	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.293	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.204	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.024	192.168.140.129	kavin	Failure
2026-07-01 21:28:44.900	192.168.140.129	kavin	Failure
2026-07-01 21:28:43.594	192.168.140.129	kavin	Success
2026-07-01 21:24:30.136	192.168.140.129	kavin	Failure
2026-07-01 21:24:30.039	192.168.140.129	kavin	Failure
2026-07-01 21:24:29.557	192.168.140.129	kavin	Failure
2026-07-01 21:24:29.417	192.168.140.129	kavin	Failure
2026-07-01 21:24:29.255	192.168.140.129	kavin	Failure
2026-07-01 21:24:29.215	192.168.140.129	kavin	Failure
2026-07-01 21:24:29.174	192.168.140.129	kavin	Failure
2026-07-01 21:24:28.968	192.168.140.129	kavin	Failure
2026-07-01 21:24:27.916	192.168.140.129	kavin	Success
2026-07-02 10:40:47.493	192.168.140.129	kavin	Success
2026-07-02 10:38:51.093	192.168.140.129	kavin	Failure

Now here is my dashboard by the name Kavin_SOC_Monitoring_Lab.

With this we can add searches now.

Using *Splunk* to Investigate a brute-force

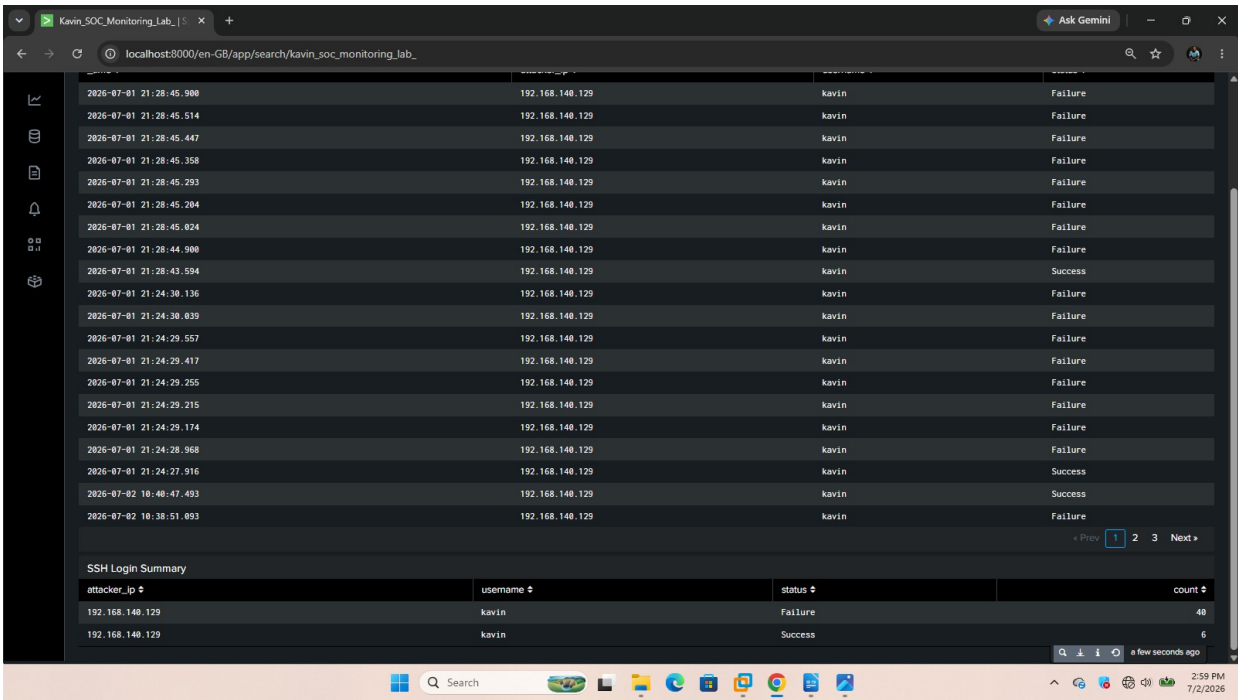
The screenshot shows the Splunk Enterprise search interface. The search bar contains the following query: `index=main "Accepted password" OR "failed password" | rex "from (?<attacker_ip>\d+\.\d+\.\d+\.\d+)" | rex "for (invalid user)?(?<username>[S+])" | eval status=if (searchmatch("Accepted"), "Success", "Failure") | stats count by attacker_ip username status`. The search results show 46 events. The table below displays the results:

attacker_ip	username	status	count
192.168.148.129	kavin	Failure	48
192.168.148.129	kavin	Success	6

Now here we separated our successful logins with failed ones and we can see that all are coming from one IP.

We can see too that our search also counts each status as we added on search *status*

Using *Splunk* to Investigate a brute-force



The screenshot shows a Splunk search interface with a table of SSH login events. The table has columns for time, IP address, username, and status. Below the table is a summary section titled 'SSH Login Summary' with a table showing counts for each IP and status combination. The interface is in dark mode and includes a sidebar with navigation icons and a bottom status bar.

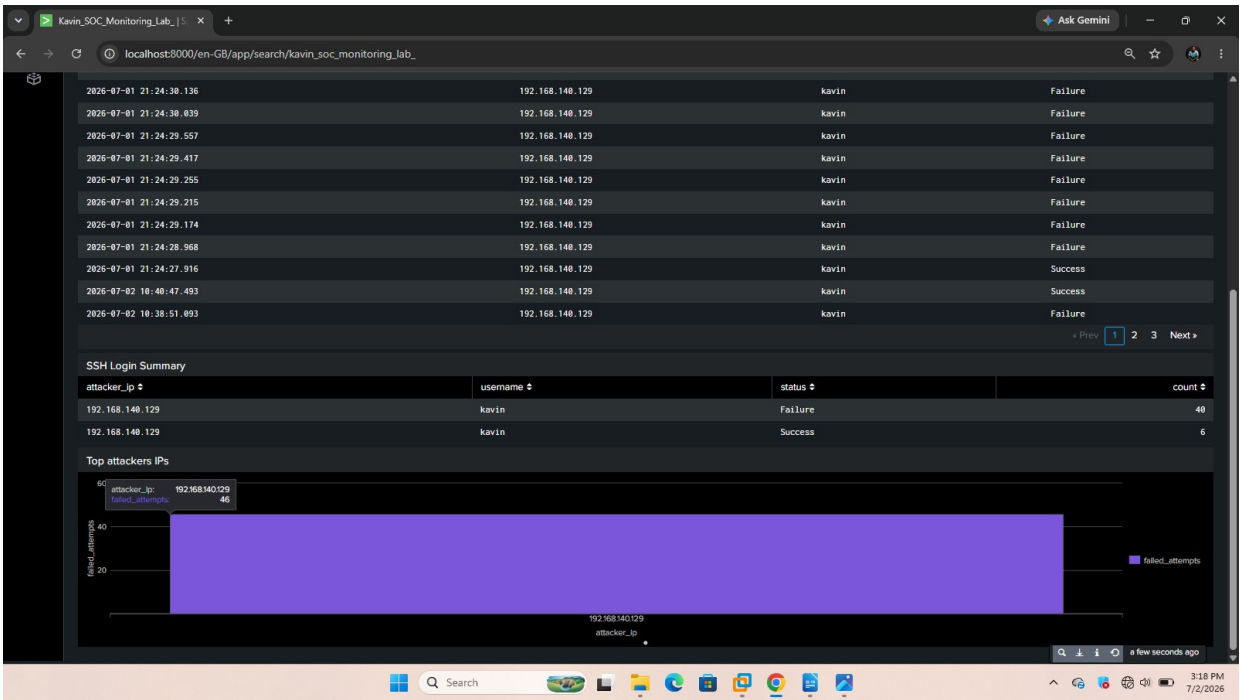
Time	IP Address	Username	Status
2026-07-01 21:28:45.908	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.514	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.447	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.358	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.203	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.204	192.168.140.129	kavin	Failure
2026-07-01 21:28:45.024	192.168.140.129	kavin	Failure
2026-07-01 21:28:44.908	192.168.140.129	kavin	Failure
2026-07-01 21:28:43.594	192.168.140.129	kavin	Success
2026-07-01 21:24:30.136	192.168.140.129	kavin	Failure
2026-07-01 21:24:30.039	192.168.140.129	kavin	Failure
2026-07-01 21:24:29.557	192.168.140.129	kavin	Failure
2026-07-01 21:24:29.417	192.168.140.129	kavin	Failure
2026-07-01 21:24:29.255	192.168.140.129	kavin	Failure
2026-07-01 21:24:29.215	192.168.140.129	kavin	Failure
2026-07-01 21:24:29.174	192.168.140.129	kavin	Failure
2026-07-01 21:24:28.968	192.168.140.129	kavin	Failure
2026-07-01 21:24:27.916	192.168.140.129	kavin	Success
2026-07-02 10:40:47.493	192.168.140.129	kavin	Success
2026-07-02 10:38:51.003	192.168.140.129	kavin	Failure

attacker_ip	username	status	count
192.168.140.129	kavin	Failure	40
192.168.140.129	kavin	Success	6

As we can see I have added a new a search on my existing dashboard and named it SSH summary, so that in future we can we can go back to it and we wont need to search same event we grab it and just use it.

Now its time I visualize my results:

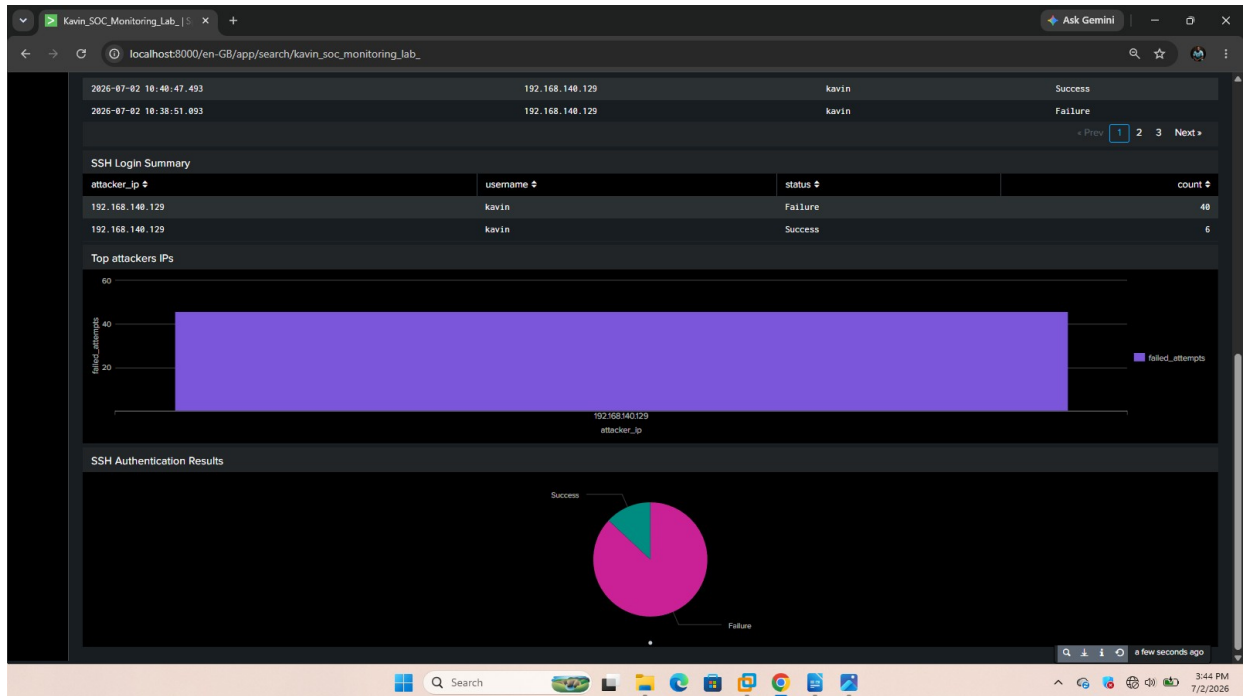
Using *Splunk* to Investigate a brute-force



Now we have added all our results as one bar to our existing dashboard

Now we separate things as real SOC:

Using *Splunk* to Investigate a brute-force



NOW as I look at the SSH Results as a Manager I can see that we had a lot of login attempts when I look at the *pie chart* but only few were successful.



Next is to investigate what made caused our system to break.

SOC Monitoring Lab Report

Project Title: SSH Brute Force Detection and Monitoring using Splunk Enterprise

Author: Kavin Kinyua

Date: 2 July 2026

Email: kavinkinyua1@gmail.com

Phone: 254118792548

1. Executive Summary

This project involved designing and implementing a Security Operations Center (SOC) monitoring solution using a home lab environment. The objective was to detect, investigate, and visualize SSH authentication activity on an Ubuntu Server using Splunk Enterprise.

The solution successfully collected system logs, identified failed and successful SSH authentication attempts, and presented security insights through a custom dashboard.

2. Project Objective

The objectives of this project were:

- Build a functional SOC monitoring environment.
 - Collect Linux authentication logs.
 - Detect SSH brute-force activity.
 - Monitor successful and failed authentication events.
 - Visualize security events using Splunk dashboards.
 - Demonstrate SOC analyst investigation techniques.
-

3. Lab Environment

Component	Purpose
Kali Linux	Security testing workstation
Ubuntu Server	Target server (UF)

Using *Splunk* to Investigate a brute-force

Component	Purpose
UFW Firewall	Host firewall
Splunk Universal Forwarder	Log forwarding
Splunk Enterprise	SIEM platform

4. Network Architecture



5. Data Collection

Authentication logs generated on Ubuntu Server were forwarded to Splunk Enterprise using the Splunk Universal Forwarder.

The monitored log sources included Linux authentication events related to SSH login activity.

6. Detection Engineering

Several Splunk Processing Language (SPL) searches were developed to support threat detection.

Examples included:

- Detection of failed SSH logins
 - Detection of successful SSH logins
 - Extraction of attacker IP addresses
 - Extraction of targeted usernames
 - Classification of authentication events as Success or Failure
-

Using *Splunk* to Investigate a brute-force

7. Investigation

The investigation focused on identifying authentication activity originating from the attacking host.

Findings

Source IP:

192.168.140.129

Target Username:

kavin

Failed Authentication Attempts:

40

Successful Authentication Attempts:

6

The successful authentication events were expected because they were generated during authorized testing within the lab environment.

8. Dashboard Development

A custom Splunk dashboard named:

Kavin_SOC_Monitoring_Lab

was developed.

Dashboard panels included:

- SSH Login Status
- SSH Login Summary
- SSH Authentication Results

These panels provide both detailed event data and summarized authentication statistics.

9. Skills Demonstrated

During this project the following cybersecurity skills were demonstrated:

- Linux Administration
 - SSH Security Monitoring
 - Log Collection
 - SIEM Operations
 - Splunk Enterprise
 - Splunk SPL
 - Log Analysis
 - Authentication Monitoring
 - Threat Detection
 - Security Dashboard Development
 - Basic Threat Hunting
-

10. Challenges Encountered

Several challenges were encountered during the project, including:

- Understanding regular expressions (*rex*)
- Learning SPL aggregation using *stats*
- Choosing meaningful dashboard visualizations
- Distinguishing between detailed event tables and summarized charts
- Understanding how grouped fields affect search results

These challenges were resolved through iterative testing and validation.

11. Lessons Learned

This project demonstrated that effective SOC dashboards should communicate security information quickly and clearly.

Rather than displaying raw logs alone, dashboards should answer key operational questions such as:

- Are authentication failures increasing?
- Which systems are under attack?
- Which accounts are being targeted?

Using *Splunk* to Investigate a brute-force

- Are authentication attempts succeeding?
-

12. Future Improvements

Future enhancements include:

- Failed Login Trend Analysis
 - Top Attacker IP Dashboard
 - Top Targeted Usernames
 - UFW Firewall Monitoring
 - Automated Brute Force Alerts
 - Email Notifications
 - Automated Incident Response
 - Threat Intelligence Integration
-

13. Conclusion

This project successfully demonstrated the deployment of a basic Security Operations Center monitoring solution using Splunk Enterprise. Authentication logs from an Ubuntu Server were collected, analyzed, and visualized to support detection of SSH authentication activity.

The project provided practical experience in SIEM configuration, log analysis, dashboard development, and basic threat detection, while establishing a foundation for more advanced SOC monitoring capabilities